



МИНОБРНАУКИ РОССИИ

**федеральное государственное автономное образовательное учреждение высшего
образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)**

Кафедра «Информационных технологий и вычислительных систем»

ЛАБОРАТОРНАЯ РАБОТА №3

«Промышленная контейнеризация приложений»

ПО ДИСЦИПЛИНЕ «Промышленное программирование»

Разработчик: аспирант Марков Д.А.

Москва, 2026 год

Цель работы

Освоить технологии контейнеризации для промышленной разработки: создание многоступенчатых Docker-образов, оркестрацию многоконтейнерных приложений с помощью Docker Compose, настройку мониторинга (Prometheus) и централизованного логирования.

Формируемые компетенции

- ПК-7.2. У-2: Работа с контейнеризацией
- ПК-7.4. У-1: Развертывание приложений
- ПК-7.4. У-2: Устранение инцидентов

Теоретическая основа

Перед выполнением работы необходимо повторить лекционный материал:

1. Архитектура Docker (образы, контейнеры, Dockerfile, реестры).
2. Многоступенчатая сборка, оптимизация слоёв, точка входа.
3. Docker Compose: сети, тома, переменные окружения, health check.
4. Мониторинг: метрики Prometheus, экспорт метрик из приложения.
5. Логирование: сбор логов через драйвер GELF, стек ELK (Elasticsearch, Logstash, Kibana).

Индивидуальное задание

Для выполнения работы предоставляется стартовый шаблон проекта (архив .zip), который можно скачать по ссылке: [\[Ссылка на шаблон\]](#). В нём готовое C++ приложение (исходный код и CMakeLists.txt), реализующее HTTP-сервер со следующими эндпоинтами:

- GET / → возвращает строку "Hello from container #N", где N – счётчик запросов (увеличивается при каждом обращении)
- GET /health → HTTP 200 (используется для health check)
- GET /metrics → возвращает метрику http_requests_total N в формате Prometheus

Вы не пишете код на C++. Ваша задача – полностью контейнеризовать это приложение и настроить инфраструктуру мониторинга и логирования в соответствии с требованиями ниже.

Практические задания

1. Создать Dockerfile для:

- базового образа (этап компиляции)
- продуктивного образа (финальный минимальный образ)

2. Настроить:

- многоступенчатую сборку (build-stage и runtime-stage)
- переменные окружения (порт приложения задаётся через APP_PORT)
- тома данных (для хранения данных Prometheus и Elasticsearch)

3. Организовать:

- сетевые взаимодействия (все сервисы в одной пользовательской сети backnet)
- сервис discovery (использовать DNS-имена сервисов в Docker Compose, например, app доступен для Prometheus как app:8080)

4. Реализовать:

- health checks (для сервиса app – проверка эндпоинта /health)
- мониторинг ресурсов (Prometheus собирает метрики с app:8080/metrics)
- логирование (логи контейнера app через драйвер gelf отправляются в Logstash, затем в Elasticsearch, просмотр в Kibana)

Требования к разработке

- Все конфигурационные файлы (Dockerfile, docker-compose.yml, prometheus/prometheus.yml, logstash/logstash.conf) должны быть написаны вручную.
- Команда docker-compose up -d --build должна запускать полностью

рабочее окружение без дополнительных действий.

- Health check должен корректно отрабатывать (контейнер app переходит в состояние healthy).
- Логи приложения (например, "Server started on port 8080") должны отображаться в Kibana.
- Метрика `http_requests_total` должна быть доступна в Prometheus.

Методические указания по выполнению

Изучите структуру полученного шаблона

Распакуйте архив с проектом ([\[Ссылка на шаблон\]](#)). В корне вы увидите папку `src/` – в ней лежит полностью готовый код C++ HTTP-сервера. Менять этот код не нужно, ваша задача – только контейнеризация.

Также в корне находятся пустые файлы, которые вы будете заполнять по ходу работы:

- `Dockerfile` – описание сборки образа;
- `docker-compose.yml` – оркестрация всех сервисов;
- `prometheus/prometheus.yml` – конфигурация сбора метрик;
- `logstash/logstash.conf` – настройка приёма и пересылки логов.

Напишите `Dockerfile` (многоступенчатая сборка)

Откройте `Dockerfile`. Нам нужно создать образ, который умеет компилировать C++ приложение (этап `build`) и потом перенести только готовый бинарник в минимальный образ (этап `runtime`). Это называется многоступенчатой сборкой – она сильно уменьшает итоговый размер.

Впишите следующие строки:

```
FROM alpine:latest AS build
```

```
RUN apk add --no-cache g++ cmake make
```

```
WORKDIR /build
```

```
COPY src/ ./
```

```
RUN cmake . && make
```

На первом этапе мы берём Alpine Linux, ставим компилятор и CMake,

копируем все исходники из src/ и собираем приложение командой `make . &&` `make`. Готовый бинарник появится в `/build/app`.

Теперь добавьте второй этап (после строки с `make`):

```
FROM alpine:latest
```

```
RUN apk add --no-cache libstdc++ curl
```

```
WORKDIR /app
```

```
COPY --from=build /build/app .
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["/app"]
```

Финальный образ снова Alpine, но уже без компилятора. Устанавливаем только `libstdc++` (нужна для работы C++ программ) и `curl` (потребуется для health check). Копируем собранный бинарник с предыдущего этапа, открываем порт 8080 и говорим, что при запуске контейнера надо выполнить `./app`. Сохраните файл. Проверьте сборку образа и запуск контейнера вручную.

Прежде чем поднимать сложное многоконтейнерное окружение, убедимся, что наше приложение вообще работает в Docker.

Выполните в терминале (находясь в корне проекта):

```
docker build -t myapp .
```

```
docker run -d -p 8080:8080 --name test-app myapp
```

Первая команда собирает образ с именем `myapp`. Вторая запускает контейнер в фоне (`-d`), пробрасывает порт 8080 с хоста на порт 8080 контейнера и даёт контейнеру имя `test-app`.

Откройте браузер и перейдите по адресу `http://localhost:8080/`. Вы должны увидеть надпись `Hello from container #1`. Если обновить страницу, цифра увеличится — значит приложение работает и счётчик запросов сохраняется внутри контейнера.

Теперь остановите и удалите тестовый контейнер, чтобы он не мешал дальнейшей работе:

```
docker stop test-app
```

```
docker rm test-app
```

Напишите базовый `docker-compose.yml` (только сервис `app`)

Docker Compose позволит нам описывать все сервисы (приложение, Prometheus, ELK) в одном файле и запускать их одной командой. Начнём с малого – опишем только наше приложение.

Откройте `docker-compose.yml`. Сначала объявите версию формата, пользовательскую сеть `backnet` (в ней контейнеры будут видеть друг друга по имени) и том `prometheus_data` (пока не используется, но пригодится позже):

```
version: '3.8'
```

```
networks:
```

```
  backnet:
```

```
volumes:
```

```
  prometheus_data:
```

Теперь добавьте сервис `app`. Обратите внимание на `health check` – Docker будет периодически проверять, отвечает ли приложение на `/health`. Если проверка не пройдёт, контейнер будет считаться `unhealthy` – это полезно для оркестрации.

```
services:
```

```
  app:
```

```
    build: .
```

```
    ports:
```

```
      - "8080:8080"
```

```
    networks:
```

```
      - backnet
```

```
    environment:
```

```
      - APP_PORT=8080
```

```
    healthcheck:
```

```
      test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
```

```
      interval: 30s
```

```
      timeout: 10s
```

```
      retries: 3
```

Запустите только этот сервис (`docker-compose up -d`).

Проверьте статус командой `docker-compose ps`. Сначала в колонке STATUS будет starting или unhealthy, но через 10–20 секунд должно появиться healthy. Это означает, что health check прошёл успешно. Если статус долго unhealthy – проверьте, что в Dockerfile вы добавили curl и что приложение действительно слушает порт 8080.

Настройте Prometheus

Prometheus – это система сбора метрик. Он будет периодически опрашивать наше приложение по эндпоинту /metrics. Нужно дать ему конфигурацию, в которой указать адрес цели.

Откройте файл `prometheus/prometheus.yml`. Впишите:

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'myapp'
    static_configs:
      - targets: ['app:8080']
```

Небольшое пояснение. `scrape_interval` – как часто собирать метрики (каждые 15 секунд). В `targets` указано имя сервиса `app` и порт 8080. Благодаря тому, что мы поместим все сервисы в одну сеть `backnet`, Prometheus сможет обращаться к `app` по этому DNS-имени.

Настройте Logstash

Logstash будет принимать логи от нашего приложения по протоколу GELF (Graylog Extended Log Format) и пересылать их в Elasticsearch.

Откройте `logstash/logstash.conf`. Впишите:

```
input {
  gelf {
    port => 12201
  }
}
```

```
output {  
  elasticsearch {  
    hosts => ["elasticsearch:9200"]  
    index => "logs-%{+YYYY.MM.dd}"  
  }  
}
```

Logstash слушает UDP-порт 12201. Любое GELF-сообщение, пришедшее на этот порт, он преобразует и отправляет в Elasticsearch, где логи будут храниться в индексах с именем типа logs-2025.04.09. Позже Kibana сможет читать эти индексы.

Дополните docker-compose.yml новыми сервисами

Теперь добавим в docker-compose.yml все недостающие сервисы: Prometheus, Elasticsearch, Logstash, Kibana. Также модифицируем сервис app – добавим драйвер логирования, который будет отправлять stdout/stderr контейнера в Logstash по GELF.

Откройте docker-compose.yml и после секции app (но внутри services:) добавьте следующие блоки. Обратите внимание на отступы (2 пробела на каждый уровень).

Сервис prometheus:

```
prometheus:  
  image: prom/prometheus:latest  
  volumes:  
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml  
    - prometheus_data:/prometheus  
  command:  
    - '--config.file=/etc/prometheus/prometheus.yml'  
    - '--storage.tsdb.path=/prometheus'  
  ports:  
    - "9090:9090"  
  networks:
```


- *backnet*

Сервис elasticsearch:

elasticsearch:

image: docker.elastic.co/elasticsearch/elasticsearch:8.6.0

environment:

- *discovery.type=single-node*

- *xpack.security.enabled=false*

volumes:

- *elasticsearch_data:/usr/share/elasticsearch/data*

ports:

- *"9200:9200"*

networks:

- *backnet*

Сервис logstash:

logstash:

image: docker.elastic.co/logstash/logstash:8.6.0

volumes:

- *./logstash/logstash.conf:/usr/share/logstash/pipeline/logstash.conf*

ports:

- *"12201:12201/udp"*

networks:

- *backnet*

depends_on:

- *elasticsearch*

Сервис kibana:

kibana:

image: docker.elastic.co/kibana/kibana:8.6.0

environment:

- *ELASTICSEARCH_HOSTS=http://elasticsearch:9200*

ports:

- "5601:5601"

networks:

- *backnet*

depends_on:

- *elasticsearch*

Важно: в сервис app добавьте следующие строки (прямо в его описание, после healthcheck):

logging:

driver: gelf

options:

gelf-address: "udp://logstash:12201"

depends_on:

- *logstash*

restart: on-failure

Пояснение:

- *logging* – переопределяет драйвер логирования. Теперь stdout и stderr контейнера будут упаковываться в GELF и отправляться на logstash:12201.
- *depends_on* – гарантирует, что logstash запустится раньше, чем app.
- *restart: on-failure* – если контейнер упадёт (например, из-за временной недоступности Logstash), Docker перезапустит его.

Также не забудьте объявить дополнительный том для Elasticsearch в самом верху файла (рядом с *prometheus_data*):

volumes:

prometheus_data:

elasticsearch_data:

Запустите полное окружение

Теперь у нас есть полностью настроенный *docker-compose.yml*. Остановите предыдущие контейнеры и удалите тома (чтобы начать с чистого листа), затем запустите всё заново:

```
docker-compose down -v
```

```
docker-compose up -d --build
```

Параметр `--build` заставляет пересобрать образ `app`, даже если он уже был.

Проверьте работоспособность

Сначала убедитесь, что все контейнеры запущены и здоровы:

```
docker-compose ps
```

Вы должны увидеть пять сервисов в состоянии `Up`. У сервиса `app` в колонке `STATUS` должно быть `healthy`.

Теперь проверьте метрики приложения:

```
curl http://localhost:8080/metrics
```

Вывод должен содержать строку `http_requests_total 0` (если вы ещё не делали запросов). Сделайте несколько запросов к `http://localhost:8080/` (обновите страницу в браузере) и снова выполните `curl` – значение счётчика должно увеличиться.

Проверьте, что `Prometheus` видит наше приложение. Откройте в браузере `http://localhost:9090/targets`. В разделе `myapp` статус должен быть `UP`. Перейдите на вкладку `Graph`, введите в поле запроса `http_requests_total` и нажмите `Execute`. Должен отобразиться график (или таблица со значением).

Теперь проверим логи в `Kibana`. Откройте `http://localhost:5601`. При первом входе выберите «`Explore on my own`». Затем откройте меню (три полоски слева) → «`Stack Management`» → «`Data Views`» → «`Create data view`». В поле «`Index pattern`» введите `logs-*` и нажмите «`Next step`». В выпадающем списке «`Time field`» выберите `@timestamp` и нажмите «`Create data view`». Теперь перейдите в «`Discover`» (в боковом меню). Вы должны увидеть записи логов – например, `Server started on port 8080` или сообщения от самого контейнера. Если логов много, обновите несколько раз страницу `http://localhost:8080/`, чтобы появились свежие записи.

Остановка и очистка

После того как вы убедились, что всё работает, вы можете остановить

контейнеры командой:

```
docker-compose down
```

При этом тома данных (метрики Prometheus, данные Elasticsearch) сохраняются. Если нужно полностью удалить всё, включая накопленные данные, выполните:

```
docker-compose down -v
```

Это полезно, когда вы хотите начать выполнение лабораторной работы заново или освободить место на диске.

Примечание: при возникновении ошибок проверьте синтаксис YAML (отступы – пробелы) и используйте `docker-compose logs <имя_сервиса>` для диагностики.

Форма отчётности

Отчёт оформляется в электронном виде (PDF) и содержит:

1. Титульный лист (ФИО, группа).
2. Цель работы.
3. Скриншот дерева папок проекта.
4. Листинги всех заполненных файлов:
 - Dockerfile
 - docker-compose.yml
 - prometheus/prometheus.yml
 - logstash/logstash.conf
5. Скриншоты:
 - Вывод `docker-compose ps` (все сервисы Up, app healthy)
 - Вывод `curl http://localhost:8080/metrics`
 - Веб-интерфейс Prometheus: график метрики `http_requests_total`
 - Веб-интерфейс Kibana: отображение логов приложения
 - Вывод `docker images` (размер финального образа)
6. Вывод.

План проведения лабораторной работы (2 занятия по 1,5 часа)

Занятие 1 (1,5 ч) – Подготовка окружения, создание Dockerfile, сборка образа

1. Приветствие, объявление темы (5 мин)

- Название лабораторной работы, цель, связь с лекционным материалом.
- Что ожидается от студентов: работающий Docker-образ приложения, многоконтейнерное окружение с мониторингом и логированием.

2. Раздача задания и шаблона (5 мин)

- Студентам выдаётся ссылка на задание (текст лабораторной работы) и архив-шаблон проекта.
- В шаблоне: папка src/ с готовым C++ кодом, пустые файлы Dockerfile, docker-compose.yml, prometheus/prometheus.yml, logstash/logstash.conf.

3. Инструкция по установке Docker (10 мин)

- Скачать Docker Desktop с официального сайта, при установке выбрать «Use WSL 2» (рекомендовано).
- После установки запустить Docker Desktop и убедиться, что работает (зелёный индикатор).
- Проверить в терминале: `docker --version`, `docker-compose --version`.
- (При необходимости) Настроить WSL2 и интеграцию с Docker Desktop.

4. Первая сборка образа (30 мин)

- Студенты пишут Dockerfile (многоступенчатая сборка на Alpine) по методическим указаниям.
- Выполняют `docker build -t myapp .` и проверяют, что образ создан. Запускают контейнер: `docker run -p 8080:8080 myapp`.
- Проверяют в браузере `http://localhost:8080/` – должно быть приветствие с счётчиком.

- Преподаватель помогает отладить ошибки (отсутствие curl, неправильный путь к исходникам, проблемы с сетью).

5. Написание docker-compose.yml (базовые сервисы) (30 мин)

- Студенты добавляют в docker-compose.yml сервис app (сборка, порт, переменная окружения APP_PORT).
- Добавляют health check для app.
- Добавляют сеть backnet и том prometheus_data.
- Запускают docker-compose up -d и проверяют, что контейнер app запускается и становится healthy.
- **Контрольная точка:** docker-compose ps показывает app в состоянии Up и healthy.
- Завершающее слово: напоминание, что к следующему занятию нужно написать конфигурации Prometheus и Logstash, а также добавить сервисы Elasticsearch, Kibana и Logstash в Compose.

Занятие 2 (1,5 ч) – Оркестрация, мониторинг, логирование, сдача работы

1. Организационный момент, проверка готовности (10 мин)

- Быстрый опрос: у кого работает app, кто добавил Prometheus и ELK.

2. Добавление сервисов мониторинга и логирования (40 мин)

- Студенты дописывают в docker-compose.yml сервисы: prometheus, elasticsearch, logstash, kibana.
- Настраивают prometheus/prometheus.yml для сбора метрик с app:8080.
- Настраивают logstash/logstash.conf (вход GELF, выход в Elasticsearch).
- Для сервиса app в Compose прописывают драйвер логирования gelf с адресом udp://logstash:12201.
- Запускают полное окружение: docker-compose up -d --build.
- **Контрольная точка:** все 5 контейнеров в

статусе Up, app – healthy.

- Проверяют доступность Prometheus (<http://localhost:9090/targets> – цель UP) и Kibana (<http://localhost:5601>).
- В Kibana создают индекс logs-* и убеждаются, что логи приложения отображаются.

3. Индивидуальная сдача работы (30 мин)

- Студенты по очереди демонстрируют преподавателю:
 - docker-compose ps (все сервисы Up, app healthy).
 - curl <http://localhost:8080/metrics> – вывод метрики.
 - Веб-интерфейс Prometheus с графиком http_requests_total.
 - Веб-интерфейс Kibana с логами приложения.
 - Вывод docker images (размер финального образа).
- Преподаватель задаёт 1–2 вопроса по теории (из списка для самоконтроля).
- Студент получает отметку о допуске и может оформлять отчёт.

4. Оформление отчёта и консультации (10 мин)

- Студенты оформляют PDF-отчёт по требованиям (листинги, скриншоты, вывод).
- Преподаватель отвечает на вопросы по структуре отчёта.

5. Завершение (5 мин)

- Подведение итогов, кто сдал, кому нужно доделать.
- Напоминание о сроке сдачи отчёта и архива.

=====

Вопросы для самоконтроля (к защите).

1. В чём отличие образа от контейнера в Docker?

Ожидаемый ответ: Образ – это неизменяемый шаблон (набор слоёв), содержащий приложение и всё необходимое для его запуска. Контейнер – запущенный экземпляр образа, который имеет своё изолированное пространство процессов, сети и файловой системы.

2. Как пробросить порт из контейнера на хост-машину?

Ожидаемый ответ: При запуске контейнера используется флаг -p хост_порт:контейнер_порт, например -p 8080:80. В docker-compose.yml это делается секцией ports: - "8080:8080".

3. Для чего нужны volumes (тома) в Docker?

Ожидаемый ответ: Тома используются для сохранения данных вне контейнера (персистентность). Они позволяют не терять данные при пересоздании контейнера, а также обмениваться файлами между контейнером и хостом.

4. Как уменьшить размер финального образа для C++ приложения?

Ожидаемый ответ: использовать многоступенчатую сборку (build-stage с компилятором и runtime-stage только с бинарником). Выбрать минимальный базовый образ (например, alpine). Объединять команды RUN в один слой и удалять временные файлы.

5. Что такое health check в Docker и зачем он нужен?

Ожидаемый ответ: Health check – это команда, которую Docker периодически выполняет внутри контейнера, чтобы проверить, работает ли приложение корректно (например, curl http://localhost/health). Если проверка не проходит, контейнер считается unhealthy, и его можно автоматически перезапустить.

6. Как в Docker Compose настроить сеть, чтобы контейнеры видели друг друга по имени сервиса?

Ожидаемый ответ: создать пользовательскую сеть (например, backnet) и подключить к ней все сервисы. Docker Compose автоматически настраивает DNS, и контейнер app становится доступен для prometheus по имени app (например, app:8080).

7. Какая команда запускает все сервисы из docker-compose.yml в фоновом режиме?

Ожидаемый ответ: docker-compose up -d. Флаг -d означает «detached» (в фоне). Если нужно пересобрать образы перед запуском, добавляется флаг --build: docker-compose up -d --build.